

Aula 2 (Módulo 01) — Permissões na prática: chmod, chown, sudo

1. Relembrando a teoria (Módulo 00, Aula 2)

Vimos que cada arquivo tem permissões `rwX` para três grupos: **dono**, **grupo** e **outros**. Agora vamos ler e alterar isso de verdade no terminal.

2. Lendo permissões com `ls -l`

```
ls -l /etc/shadow
```

Saída típica:

```
-rw-r----- 1 root shadow 1234 Set 10 10:00 /etc/shadow
```

Decompondo caractere por caractere:

```
- rw- r-- ---
|  |  |  |
|  |  |  |  └─ outros: sem nenhuma permissão
|  |  └───┬── grupo (shadow): pode ler
|  └───┬───┬── dono (root): pode ler e escrever
└───┬───┬───┬── tipo do arquivo ('-' = arquivo comum; 'd' = pasta)
```

Isso confirma na prática o que discutimos na Aula 2 do Módulo 00: o arquivo de senhas do Linux só é legível pelo `root` e pelo grupo `shadow` — um usuário comum não consegue ler, e é exatamente por isso que sua tentativa de `cat /etc/shadow` como usuário comum falha com "Permission denied".

3. `chmod` — alterando permissões

`chmod` = *change mode*. Duas formas de usar:

Forma numérica (octal)

Cada permissão vira um número: `r=4`, `w=2`, `x=1`. Soma-se para cada grupo (dono, grupo, outros):

```
chmod 750 script.sh
```

7 (dono: `4+2+1` = `rwX`) 5 (grupo: `4+0+1` = `r-x`) 0 (outros: nada).

Forma simbólica

```
chmod u+x script.sh      # adiciona (+) execução (x) para o dono (u=user)
chmod g-w arquivo.txt    # remove (-) escrita (w) do grupo (g)
chmod o=r arquivo.txt    # define (=) "outros" (o) como só leitura (r)
```

Por que importa para segurança: permissão errada é uma das causas mais comuns de incidente — um arquivo de configuração com senha em texto puro, legível por "outros" (`chmod 644` ou pior), é uma falha real e comum. Do lado ofensivo, um atacante que consegue tornar um arquivo executável (`chmod +x`) em uma pasta gravável (como `/tmp` , que vimos na aula passada) está a um passo de rodar código arbitrário.

4. `chown` — trocando o dono

```
sudo chown ana:devs arquivo.txt
```

Isso muda o **dono** para `ana` e o **grupo** para `devs` . Precisa de privilégio elevado (por isso o `sudo` — veja abaixo) para mudar o dono de um arquivo que não é seu.

Por que importa para segurança: um arquivo malicioso "pertencendo" a um usuário do sistema em vez do usuário comum que o baixou é um detalhe que aparece em investigações — o dono do arquivo ajuda a reconstruir **quem** o colocou ali.

5. `sudo` — executando como outro usuário (normalmente root)

`sudo` = *substitute user, do* (ou popularmente "superuser do"). Permite que um usuário autorizado execute um comando **como se fosse outro usuário** — normalmente `root` — sem precisar fazer login como `root` diretamente.

```
sudo apt update
```

Quem pode usar `sudo` , e para quais comandos, é controlado pelo arquivo `/etc/sudoers` (editado com o comando `visudo` , nunca editando o arquivo direto — isso evita erros de sintaxe que travam o sistema) e pela associação a grupos como `sudo` (Ubuntu/Debian) ou `wheel` (RHEL/Fedora), que mencionamos no Módulo 00, Aula 2.

Por que importa para segurança: todo uso de `sudo` fica registrado em log (geralmente em `/var/log/auth.log` no Ubuntu/Debian, ou `/var/log/secure` em RHEL/Fedora) — quem executou, qual comando, quando. Esse é um dos primeiros lugares que um analista olha ao investigar **escalada de privilégio**: um usuário comum de repente usando `sudo` para rodar algo fora do seu padrão normal de trabalho é um sinal de alerta. Configuração incorreta do `sudoers` (dar permissão de `sudo` sem senha, ou para comandos perigosos demais) é uma das causas mais comuns de escalada de privilégio real em ambientes Linux.

6. Exercício prático

```
cd /tmp
touch meu_script.sh
ls -l meu_script.sh          # observe as permissões padrão
chmod 750 meu_script.sh
ls -l meu_script.sh          # compare: o que mudou?
echo 'echo "rodei com sucesso"' > meu_script.sh
./meu_script.sh               # deve funcionar (dono tem x)
chmod u-x meu_script.sh
./meu_script.sh               # agora deve falhar (Permission denied)
rm meu_script.sh
```

Observe a mensagem de erro na última execução — "Permission denied" é uma das mensagens mais comuns em investigação (tanto do lado "isso não deveria acontecer" quanto do lado "que bom que o controle de acesso funcionou").

7. Recapitulando

- `ls -l` lê permissões (`rwX` para dono/grupo/outros) — já conecta direto com a teoria do Módulo 00.
- `chmod` altera permissões (numérico `750` ou simbólico `u+x`).
- `chown` altera o dono/grupo de um arquivo.
- `sudo` executa como outro usuário (normalmente root), é controlado por `/etc/sudoers` , e **todo uso fica logado** — um dos primeiros lugares a olhar ao investigar escalada de privilégio.

Próxima aula

Processos, serviços e `systemd` na prática — vamos usar `ps` , `top` e `systemctl` de verdade para observar o que aprendemos na teoria (Módulo 00, Aulas 1 e 3), e começar a olhar para `cron` como mecanismo de persistência.

Fontes recomendadas

- **Linux man-pages** — `chmod(1)` , `chown(1)` , `sudo(8)` : referência oficial de cada comando.
<https://man7.org/linux/man-pages/man1/chmod.1.html>
- **Sudo Project** — **documentação oficial do `sudoers`** : como as regras de `sudo` são definidas e auditadas.
<https://www.sudo.ws/docs/man/sudoers.man/>